

Week4

Task1:

Write a C program where a parent process creates multiple child processes and synchronizes their completion using `wait()` and `waitpid()`. Compare the behavior of both functions

Source Code:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
int main(){
    int n = 3;
    pid_t pid;
    printf("Parent process started. PID = %d\n", getpid());
    for (int i = 1; i <= n; i++) {
        pid = fork();
        if (pid < 0) {
            perror("fork failed");
            exit(1);
        }
        if (pid == 0) {
            printf("Child %d: PID = %d, Parent PID = %d\n",
                i, getpid(), getppid());
            sleep(i);
            printf("Child %d completed.\n", i);
            exit(0);
        }
    }
    printf("\nParent waiting for all children using wait()...\n");
    for (int i = 0; i < n; i++) {
        wait(NULL);
    }
    printf("All child processes completed.\n");
    printf("\nDemonstrating waitpid()...\n");
    for (int i = 1; i <= n; i++) {
        pid = fork();
        if (pid < 0) {
            perror("fork failed");
            exit(1);
        }
        if (pid == 0) {
            printf("Child %d created using waitpid(): PID = %d\n",
                i, getpid());
            sleep(i);
            printf("Child %d completed.\n", i);
            exit(0);
        }
    }
}
```

```

    }
    printf("All child processes completed.\n");
    printf("\nDemonstrating waitpid()...\n");
    for (int i = 1; i <= n; i++) {
        pid = fork();
        if (pid < 0) {
            perror("fork failed");
            exit(1);
        }
        if (pid == 0) {
            printf("Child %d created using waitpid(): PID = %d\n",
                i, getpid());
            sleep(i);
            printf("Child %d completed.\n", i);
            exit(0);
        }
    }
    for (int i=0;i<n;i++) {
        waitpid(-1,NULL,0);
    }
    printf("All child processes completed using waitpid().\n");
    return 0;
}

```

Output:

```

Parent process started. PID = 515
Child 1: PID = 516, Parent PID = 515
Child 2: PID = 517, Parent PID = 515

Parent waiting for all children using wait()...
Child 3: PID = 518, Parent PID = 515
Child 1 completed.
Child 2 completed.
Child 3 completed.
All child processes completed.

Demonstrating waitpid()...
Child 1 created using waitpid(): PID = 519
Child 3 created using waitpid(): PID = 521
Child 2 created using waitpid(): PID = 520
Child 1 completed.
Child 2 completed.
Child 3 completed.
All child processes completed using waitpid().

```

Task2:

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

Source Code:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0) {
        printf("Child process created. PID = %d\n", getpid());
        printf("Child process is exiting...\n");
        exit(0);
    } else {
        printf("Parent process. PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);

        sleep(5);

        printf("Parent calls wait() to eliminate zombie process.\n");
        wait(NULL);

        printf("Child process has been reaped successfully.\n");
    }

    return 0;
}
```

Output:

```
Parent process. PID = 871
Child PID = 872
Child process created. PID = 872
Child process is exiting...
Parent calls wait() to eliminate zombie process.
Child process has been reaped successfully.
```